



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

University Tunku Abdul Rahman

Faculty of Information and Communication Technology

(Kampar Campus)

UCCE3073 Computer Vision and Image Processing

Dr. Muhammad Syaiful Amri Bin Suhaimi

June 2025

Individual Report: Dog-Cat Breed Recognition System

Name: Chee Hao Jie

Student ID: 2103820

in CT programme

Table of Contents

Project Overview	4
Part 1 Face Authentication	5
1.1 Objective	5
1.2 Literature Review	5
1.3 Methodology	8
1.3.1 Concept of Face Embeddings	8
1.3.2 Generate Face Embeddings	9
1.3.3 Compare Face Embeddings	9
1.3.4 Image Preprocessing	9
1.3.5 Database Preparation Algorithm Flow	10
1.3.6 Face Authentication Algorithm Flow	11
1.4 Assumptions and Constraints	12
1.4.1 Assumptions	12
1.4.2 Constraints	12
1.5 Result and Discussion	13
1.5.1 GUI Display	13
1.5.2 Image Preprocessing in Google Colab	14
1.5.3 Authentication Result in Google Colab	15
1.5.4 Face Detection in Application	17
1.5.5 Authentication Result in Application	19
1.6 Summary and Future Improvement	20
1.6.1 Summary	20
1.6.2 Future Improvement	21
Part 2: Pet Classification	22
2.1 Objective	22
2.2 Literature Review	22
2.3 Methodology (Algorithm Flow)	24
2.3.1 Models Preparation Algorithm Flow	24
2.4 Assumptions or Constraints made	27
2.4.1 Assumptions	27
2.4.2 Constraints	27

2.5	Result and discussion	28
2.5.1	Model Performance and Hyperparameter Tuning	28
2.5.2	GUI Display	36
2.5.3	Classifications Results in Application	38
2.6	Summary and Future Improvements	41
2.6.1	Summary	41
2.6.2	Future Improvements	42
	References	43

Project Overview

This project presents a graphical user interface (GUI) based application that is able to recognize the user identity and classify dog and cat breeds. The development of this application involves two main parts including face authentication and dog/cat breed classification features.

1. Face Authentication

When the user first start up the program, the system will first verifies the identity of the user by capturing and comparing it to the image that store in the database. Only the authenticated user are granted access to proceed to the next stage of the application.

2. Pet Classification

The pet classification model allow user to classify two categories of pet which are dog and cat. User can upload the pet image, and the model will classify the breed and display the breed with highest confidence.

The remaining of this project is divided into two main sections including:

- **Part 1: Face Authentication**
- **Part 2: Pet Classification**

Part 1 Face Authentication

1.1 Objective

The face authentication feature serve as a safety measurement to ensure only the authenticate user is able to access the application. This feature offer a modern approach to classify the user identity instead of relying on traditional authentication method like enter password or PIN code. Since all individuals have unique facial characteristic, thus face authentication provide a more secure and effective way to safeguard the user access.

The goals of this part are:

1. To develop a face detection system that is able to identify human face
2. To convert the detected face into face embeddings and store as a reference
3. To accurately authenticate the user image by comparing its face embeddings with stored reference images.
4. To integrate the authentication system into a self-developed GUI application.

1.2 Literature Review

Dynamic Face Authentication Systems: Deep Learning Verification for Camera Close-Up and Head Rotation Paradigms.

Castelblanco and his team [1] looked at a different way of handling face authentication by moving away from the usual static image approach. Instead of just comparing a still photo of someone's face, they designed a system where users are asked to perform small movements like leaning closer to the camera or turning their head slightly. These actions give the system extra spatial details that can be picked up by deep learning models, in this case CNNs and a Siamese network.

To test the idea, they built their own dataset, which included both real users and spoofing attempts such as printed photos and video replays. The results were very strong: the system could detect whether someone was real or fake with around 97–98% accuracy, and it worked well for both close-up and head-turning challenges. What makes this study important is that it tackles one of the biggest weaknesses of face authentication—spoofing. The challenges are when users respond to simple prompts, the system becomes much harder to trick with photos

or videos. It shows how adding a dynamic element can make face recognition not just accurate but also more secure.

Combining CNN, Attention, and BiLSTM for Face Recognition

Qi et.al [2] proposed new approach of deep learning model for face recognition, which they call AB-FR. At its core it's a CNN, but they added two important parts: an attention mechanism and a BiLSTM layer. The attention module helps the network focus on the most useful features in an image, while the BiLSTM brings in a way to understand sequences, like frames from a video or images taken from slightly different angles.

They tested out the model within well-known datasets like CASIA-FaceV5, LFW, and also ORL, and the impressive performance showed such consistently high results all across the board, with over 99% accuracy seen on some datasets. The system was prominent for its strength and robust even throughout challenging lighting or head pose changes. Face authentication research is heading toward models combining spatial detail with temporal information instead of just a single image, so this work matters. For actual applications, this means a better level of reliability when the input images happen to be imperfect, which often occurs outside of controlled lab conditions.

Recognizing Faces with Masks Using FaceMaskNet-21

The common use of face masks during the COVID-19 pandemic created problems for traditional face recognition systems. Golwalkar with Mehendale [3] introduced FaceMaskNet-21 to address this issue. It is a small neural network designed especially for faces wearing masks. The system learns embeddings that are compact (feature vectors of 128 dimensions) capturing parts of the face that are visible, instead of features hidden that it reconstructs.

The model achieved near to 89% accuracy even with occlusion that was caused by masks as well as processed images fast in 10 milliseconds. That becomes practical in real-time scenarios like login systems or security checkpoints. This study has value because of how it authenticates faces which are showing adaptations for changes in the real world. Deep learning models' flexibility was highlighted as authors focused on masked-face recognition. Older biometric systems simply weren't built under such conditions. That research is a good example since it meets urgent, practical needs and still contributes to secure authentication.

Summary of Literature Review

The three reviewed studies show just how face authentication improved in different ways in recent times. [1] stopped spoofing attacks via dynamic challenges like head movements. [2] improved accuracy through CNNs combined with BiLSTM and attention. The models capture changes across multiple images along with detailed features. By the use of deep metric learning, [3] solved the masked faces problem via generating strong embeddings as well as with occlusions.

Our method is simpler compared to these methods, but it still follows along with the same basic idea: we detect all of faces, improve upon them, generate more embeddings, and then compare them through cosine similarity. [3] work informs matching related to embeddings. We depend also upon them. Our method focuses upon the making of features that are more discriminative, similar to [2]. [1] liveness check is what it is that we don't yet have, and it could be a useful sort of improvement in the future.

1.3 Methodology

1.3.1 Concept of Face Embeddings

In this project, face embeddings method is used to design face authentication systems. Face embeddings is a modern approach to authenticate face. Basically embedding is a vector of number that is encoded based on the unique features of a face image. Rather than simply comparing the raw images, this method convert each face into numeric representation in a feature space [4]. In other word, the similarity between two corresponding faces can be evaluated by the distance between two vectors.

$$f\left(\text{Image of a man}\right) = \begin{pmatrix} 0.112 \\ 0.067 \\ 0.091 \\ 0.129 \\ 0.002 \\ 0.012 \\ 0.175 \\ \vdots \\ 0.023 \end{pmatrix}$$

The reason why embeddings are suitable to verify a person identity is because every human have unique face features. For example, suppose two images of same person are taken under different conditions like different angles, expression or lighting. If these two images are map to vector embeddings, their vector must be close together. On the other hand, if the vector are compare with totally different people, then the vector must be far apart. This property offer an efficient and reliable way to perform identity verification. As described by [4], this method generalizes the idea of representation learning where the system learns what features are important and encodes them into a more compact and machine-readable format.

The advantage of this approach is that once embeddings are created, face authentication are more towards to vector similarity problem where can be solved by using mathematical distance measures. This make the system more efficient and faster than other pixel-based approach like Convolutional Neural Network method.

1.3.2 Generate Face Embeddings

The process of generate embeddings typically involves deep neural network where a large dataset of human face images are used. The architecture like CNN are widely used, with popular models like FaceNet, VGGFace and ArcFace. During training, the network learns to map input images into a lower-dimensional space where the embeddings preserve identity relationships.

In this project, embeddings are generated using the imgbeddings library which provides an efficient way to produce fixed length vectors for each detected face. Before the embeddings are generated, preprocessing steps are taking place. In this case, the system will first detect the face, crop the face and enhance the face image. These steps are crucial to ensure the generated vector embeddings are consistent.

1.3.3 Compare Face Embeddings

Once embeddings are generated for both user face (captured from camera) and reference user image (stored in database), the system needs to compare them to determine if they belong to the same person. In this case, the similarity is measured using cosine similarity formula.

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{|A| |B|}$$

where

A is vector of user face

B is vector of reference user image

If the value is close to 1.0, then the vectors are pointing near to the same direction indicates that the faces are very similar. On the other hand, if value is close to 0, it means the vector are in the opposite direction. Thus, faces are highly unrelated.

1.3.4 Image Preprocessing

Image preprocessing is a crucial step to ensure the quality of image before generating the face embedding. The preprocessing step aims to reduce the image noise and enhance the features to make the embeddings vector more reliable.

In this project, four preprocessing step involves:

- **RGB Conversion**

All the image is process in RGB format as most pre-trained face embedding model including the imgbeddings library are trained in RGB images. Converting to grayscale image will reduce the accuracy as the model expects three input channels.

- **Face Detection**

The face detection involve using dlib frontal face detector(), it is a pretrained library that use to identifies the location of a face in the image.

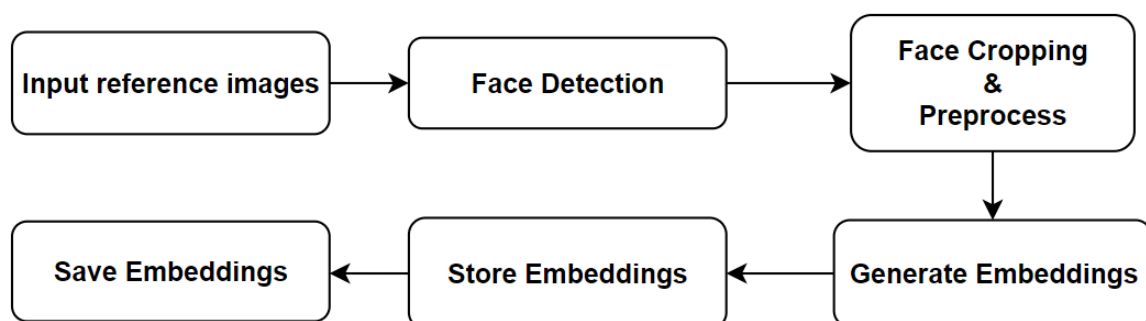
- **Cropping**

The detected face is cropped with a margin to include facial features and reduce the background interference.

- **Enhancement**

The cropped image then undergoes enhancement to adjust the sharpness, contrast and brightness to improve the image quality.

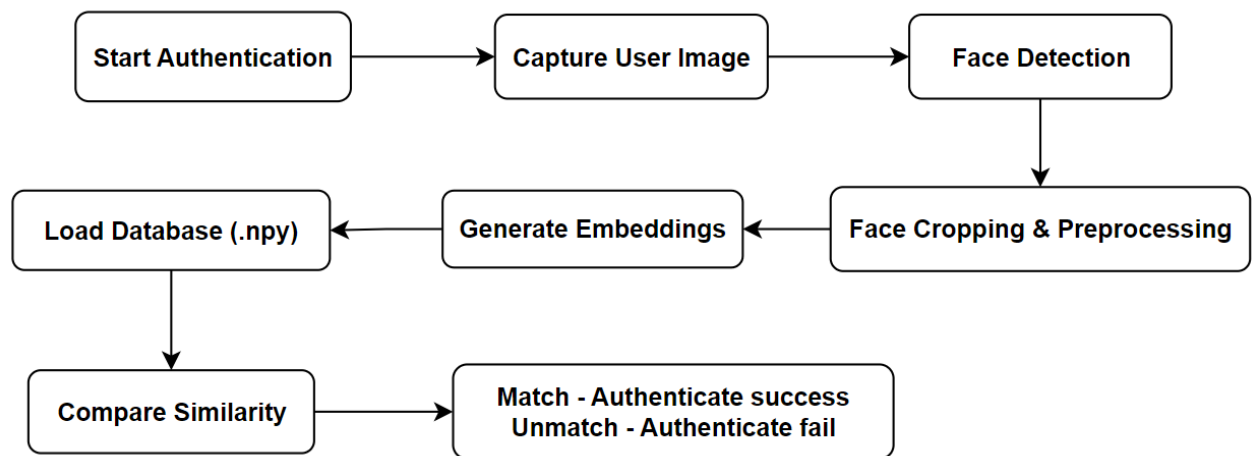
1.3.5 Database Preparation Algorithm Flow



Above is the algorithm flow to create reference database of the authorized users. The process begins with collecting the user images. Each image will undergoes face detection to locate the face, followed by cropping and preprocessing. During image preprocessing, the image will run through RGB conversion, enhance sharpness, enhance contrast and adjust brightness. After enhancement, the face is then converted into an embedding vector using imgbeddings and

stored in an array. Finally all the vector embeddings are saved in a single .npy file. The face database for the user is now ready.

1.3.6 Face Authentication Algorithm Flow



The above flow demonstrate how user is authenticated when using the application. When the authentication begins, the camera will start capture user image and applies the same preprocessing steps as shown in database preparation. After the face embeddings for detected face is generated, the system loads the stored embeddings from the .npy file and compares them with the embedding generated just now using cosine similarity. If the similarity scores exceeds the predefined threshold, then the user is granted access, or else access denied.

1.4 Assumptions and Constraints

1.4.1 Assumptions

1. **Face Visibility:** The user's face is clearly visible to the camera without distractions like mask, sunglass, hat or partially closed by hand or hair.
2. **Lighting Conditions:** All images are captured under normal lighting, ensuring facial features can be detected.
3. **Frontal Faces:** User are expected to face the camera in proper position, ensure the camera can clearly detect the face.
4. **Single Authorization:** The system is developed to authorized one person per time. If multiple face appear within the camera frame, only the first detected face is taken account.
5. **Threshold:** The similarity threshold is set to 0.90, which is assumed to be sufficient enough to distinguish the valid user.

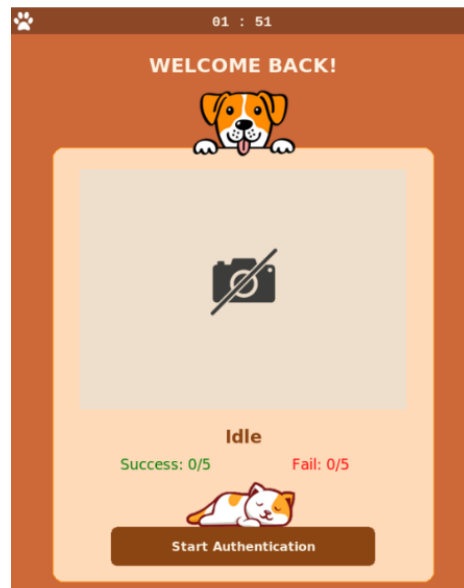
1.4.2 Constraints

1. **Limited Database Size:** The system stores embeddings in .npy file, which only work efficiently for small group of users. But as more images are stored in the database, the application tends to become slow and inefficient.
2. **Environment Dependencies:** The system strongly relies on camera to detect user face. Thus, performance downgraded under extreme condition like poor lighting, unnatural head pose or partial occlusions.
3. **Real-Time Processing Limitation:** Since face detection, preprocessing, and generate embedding need computation time, thus the application need to run in low frame rate for better performance.

1.5 Result and Discussion

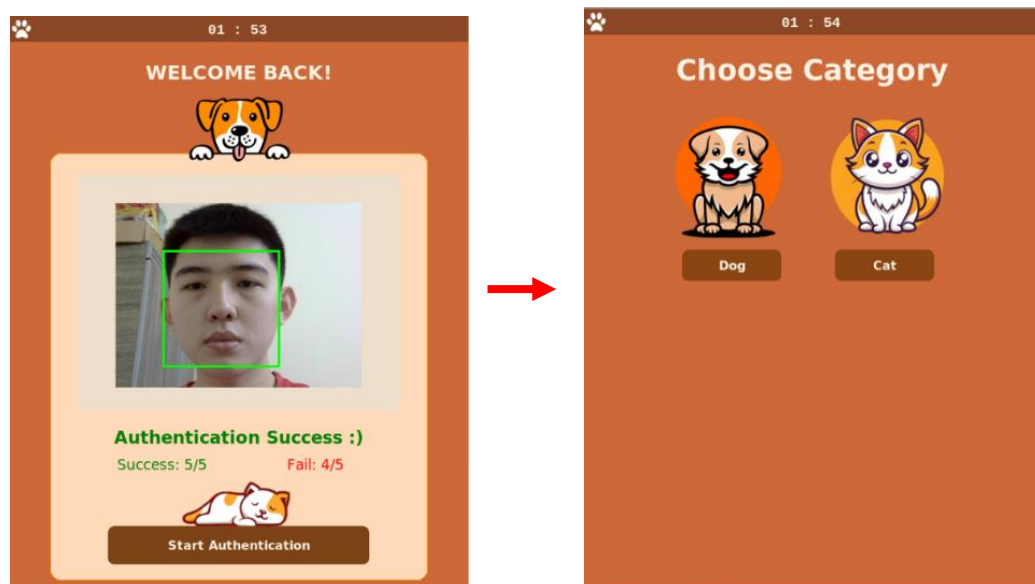
1.5.1 GUI Display

Authentication Page



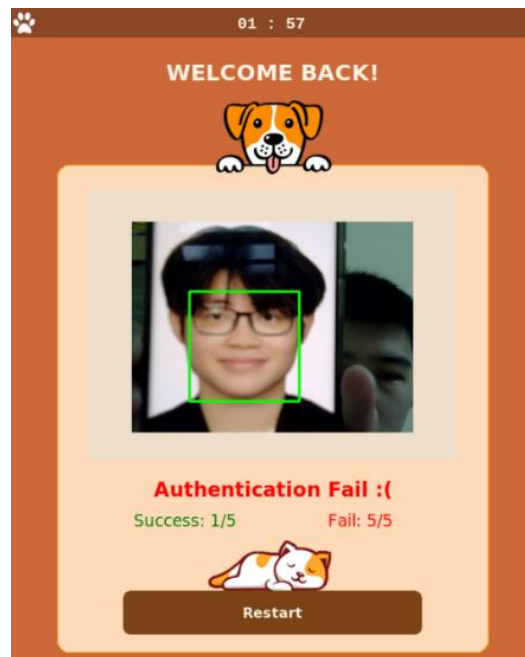
The application begin authentication page. Since the theme of the application is pet, thus beautiful image such as dog and cat are used to design the interface. The no camera icon in the middle is to indicate no operating camera currently. Also to make the interface more user friendly, button widget are used to prompt user interaction. To initiate the authentication process, user may need to press the Start Authentication button.

Authentication Success



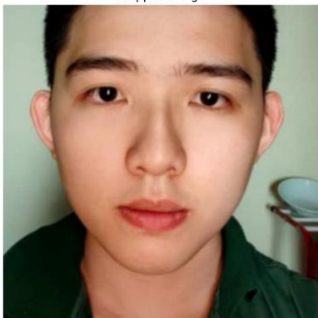
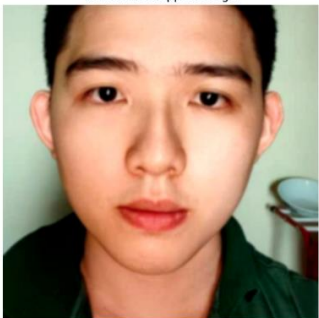
When the button is pressed, the camera will start capture the frame. When the user passed all five attempts, feedback of “Authentication Success :)” will appear on the GUI indicate he is the valid user. Then, user will redirect to the pet classification page shortly.

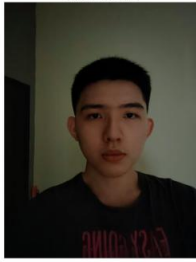
Authentication Fail

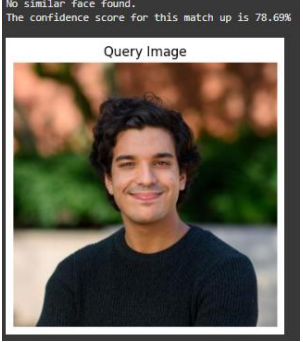
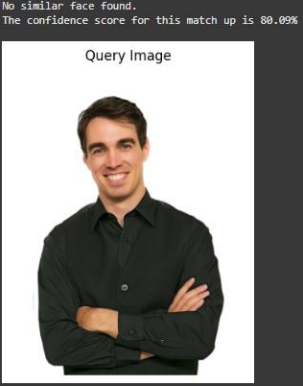


When user fail for all five attempts, feedback of “Authentication Fail :(” will appear on the GUI indicate he is invalid user. Then, user may need to press the Restart button to start another round of authentication.

1.5.2 Image Preprocessing in Google Colab

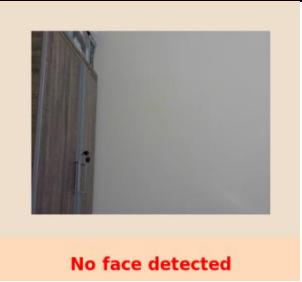
No	Original Image → Cropped Image → Enhanced Cropped Image		
1	Original Image 	Cropped Image 	Enhanced Cropped Image 

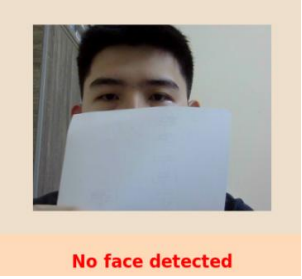
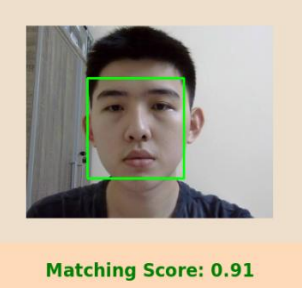
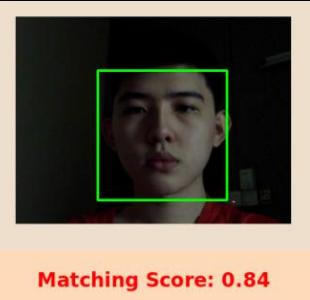
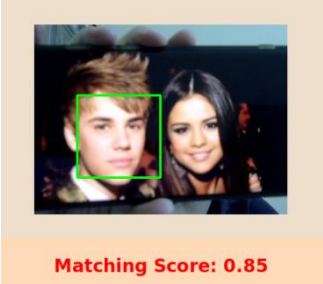
2	We found a match. The confidence score for this match up is 94.49% Query Image Database Matched Image (face1.jpg)  	94.49	Pass
3	No similar face found. The confidence score for this match up is 72.12% Query Image 	72.12	Fail
4	No similar face found. The confidence score for this match up is 79.12% Query Image 	79.12	Fail
5	No similar face found. The confidence score for this match up is 77.34% Query Image 	77.34	Fail
6	No similar face found. The confidence score for this match up is 77.07% Query Image 	77.07	Fail

7		78.69	Fail
8		80.09	Fail

The face authentication robustness in Google Colab is evaluated by uploading the image. This is because Google Colab does not support open cam feature. The overall results are accurate where the system is able to identify the valid user correctly. For valid user (Image 1 and Image 2), the system consistently achieved a high confidence level, which score even higher than the predefined threshold (90%). While the confidence of other users are mostly less than 80%. This indicates that the embeddings vector that map the facial features are accurate and able to differentiate between different individuals.

1.5.4 Face Detection in Application

No	Sample	Face Detected?
1		No

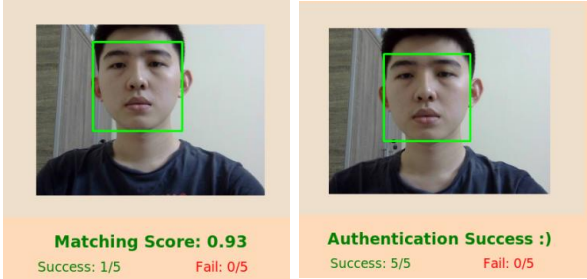
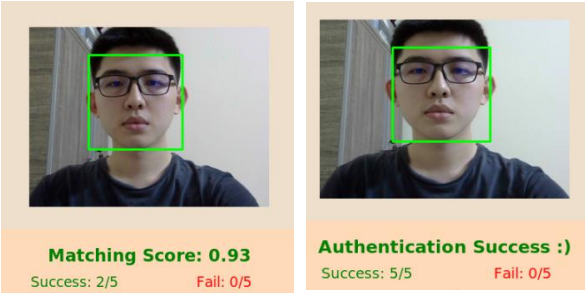
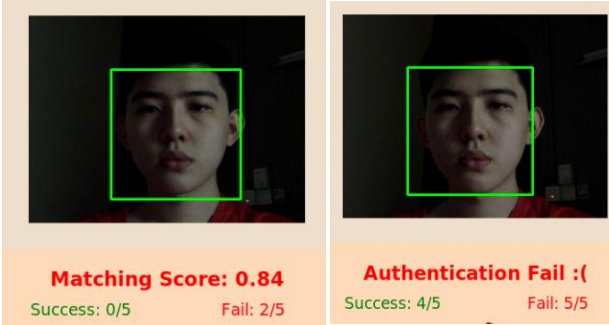
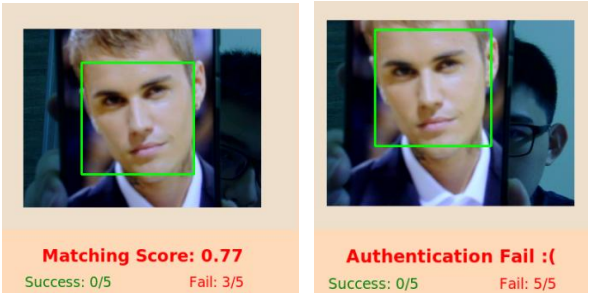
2		No
3		Yes
4		Yes
5		Yes

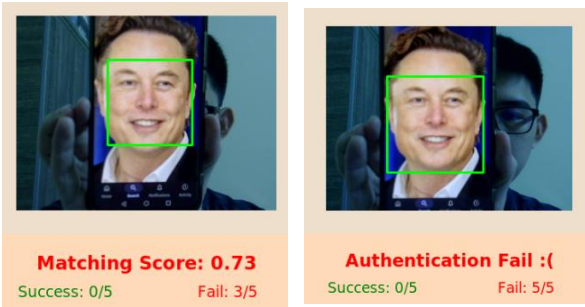
In the GUI, when the user press the start authentication button, the system will first detect face before authenticate the user. Based on the result obtained, when no valid face is detected within the frame, we may observed the feedback, the system will output “No face detected” in red color. By observing image 2, if the user face is partially closed by obstacles, the system will also classify it as no face detected.

When face is being detected, a green square box will appear, and the feedback will show the confidence score of the detected face in green color as shown in image 3. Lastly for image 3, when multiple faces are detected, only the first detected face is taken account and undergoes

the authentication process. Overall the system is robust where the authentication will only begin after face is being detected.

1.5.5 Authentication Result in Application

No	Sample	Result (>90%)
1		Pass
2		Pass
3		Fail
4		Fail

5		Fail
---	--	------

Based on the results obtained from application interface, it is proven that embedding-based face authentication is a practical and effective approach to authenticate an individual under controlled condition. The system consistently achieved a high confidence level in identifying the user and reject all invalid user. However, the system sometimes will fail to authenticate the user if the environment condition is poor. For example, as shown in image 3, the light condition is poor leading to fail authentication.

For GUI performance, by using lower frame rate, the real-time detection run smoothly without lag. Thus the system is able to detect the user face, calculate the confidence score, and redirect smoothly to pet classification page without delay. This responsiveness makes the face embedding method practical in real-world application as efficiency and user experience are important.

1.6 Summary and Future Improvement

1.6.1 Summary

As a summary, the face authentication system successfully shows that embedding-based verification is effective in recognizing the user. The system is able to authenticate users with high accuracy by combining face detection, image preprocessing, embedding generation, as well as cosine similarity comparison. Other than that high confidence score confirms beyond doubt that the approach is reliable. With brightness, contrast, and sharpness adjustment, the preprocessing step further improved the clarity of cropped images so that stronger embeddings are mapped. As both Google Colab as well as the GUI application interface implemented the same approach, it is proven that it is capable of operating in real time, making it practical for user authentication.

1.6.2 Future Improvement

Although the embedding-based face authentication system show promising results, some limitations were found at the testing and deployment levels. First, the system presently has no liveness check and hence is incapable of distinguishing between an actual user and a spoof attack like a printout of a face or video replay. This diminishes the system robustness in real-world applications. This limitation could be addressed in future work by incorporating liveness detection techniques, for example blink detection, random head-movement prompts, or the use of infrared and depth sensors to distinguish live users from fraudulent inputs.

Secondly, system performance is very environment dependent. While it performs well with frontal well-illuminated images, performance is poor when faces are partially covered by objects like masks or spectacles or hands or when low-illumination and extreme face pose images are applied. To address this issue, future improvements could involve the use of improved preprocessing methods for illumination normalization as well as the use of data augmentation to better simulate real-world scenarios in addition to more strong deep learning models trained on diverse conditions.

In terms of scalability, the system keeps embeddings in a single .npy file, which is ideal for small datasets. However, for large databases in which quite fast retrieval and even updating are needed, that system will be very inefficient. Practically, integrating a more scalable storage solution such as a relational database as well as a specialized vector database like FAISS would improve efficient indexing and retrieval.

Part 2: Pet Classification

2.1 Objective

The pet classification model is the second stage of the application. The main objective of this project is to develop a classification model which is capable of classify both dog and cat breeds by using deep learning models. This model is unlike the binary classification where only differentiate between cats and dogs instead it focuses on identifying the exact breed of each species.

The goals of this part are:

1. To build and train a dog breed classification model that distinguish between multiple dog breeds.
2. To build and train a cat breed classification model to distinguish between multiple cat breeds.
3. To apply transfer learning with pre-trained CNN architectures including InceptionV3 and EfficientNetB4 for efficient training on limited dataset.
4. To evaluate each model independently using metrics such as accuracy, precision, recall and confusion matrix.
5. To deploy the models in lightweight TensorFlow Lite format which is suitable for real-time interface.
6. To integrate both models into a self-developed GUI application which was built using Tkinter.
7. To utilize OpenCV that allows live classification of either cat breeds or dog breeds.

2.2 Literature Review

Dog Breed Classification

Several studies have applied deep learning to fine-grained dog and cat breed classification. In [5], the authors investigated fine-grained dog breed recognition because they used deep convolutional neural networks. Their work focused on two architectures Beginning-ResNet as well as NASNet-A mobile, within which they were applied to the Stanford Dogs dataset that

contains 120 breeds. Beginning-ResNet achieved an accuracy of 90.7% while mobile NASNet-A had 80.7% in accuracy, performing worse. One strength of this study is that it demonstrated early transfer learning for large-scale breed recognition also showed ImageNet pre-trained models adapt well to fine-grained classification tasks. Its practical implementation provides another strength in that the authors developed a mobile application and client-server for the deployment. However, reliance that is on older architectures and failure when it comes to augmenting data as well as inability for addressing class imbalance limited the study.

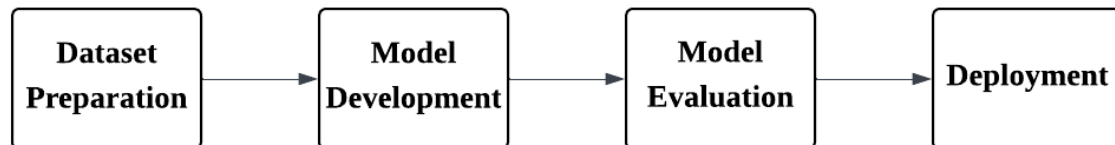
[6] helped this field when a hybrid approach was proposed by them. The approach had combined both of InceptionV3 networks with Xception networks. Each model extracted features independently which were then concatenated and were fed into a joint classifier. On the Stanford Dogs dataset, the hybrid model achieved an accuracy of 92.4%. It outperformed individual networks plus other hybrid combinations that were tested like ResNet50+InceptionV3 and EfficientNetB3+Xception since it was the better one. This new work was strong because it fused feature representations, and it enabled great accuracy. However, since it relies upon the two deep architectures, it demands meaningful computation. This reliance and also the high computational demands may limit deployment in real-time or resource-constrained environments.

Furthermore, according to [7], the study approached dog breed classification from the perspective of class imbalance. By using a dataset of 133 breeds with over 20000 images, they data augmentation is applied to balance minority classes. Then they developed a stack ensemble combining InceptionV3 and VGG16. The stacked model achieved a test accuracy of 88.7%, which improved from single InceptionV3 which had an accuracy of 84.4% and accuracy of 73.4% from VGG16. The main contribution of the study was its explicit focus on class imbalance, which is often overlooked in fine-grained recognition. The stacked ensemble also showed improved generalization over single models. However, this approach comes with drawback which consist of significant overfitting, with training accuracy reaching 99% while test accuracy only 88.7%. Furthermore, augmentation applied uniformly across classes degraded the performance rather than improving it. In compared to the hybrid methods proposed in [5], the stacked ensemble was less accurate and more computationally heavy without yielding state-of-the-art results.

2.3 Methodology (Algorithm Flow)

2.3.1 Models Preparation Algorithm Flow

The methodology for this project consists of four stages which were dataset preparation, model development, model evaluation and deployment. Separate models were developed for dog and cat breeds classification, but both of the models follow the same pipeline.



Step 1: Dataset Preparation

First, the image datasets for both dog and cat breeds were obtained from the Kaggle repositories. The original dog dataset consist of 120 breeds. To ensure training efficiency, only 20 dog breeds are selected for dog classification model. While for cat dataset, 9 are selected out of 12 breeds for the cat classification model. After selecting the datasets, the dataset was divided into training, validation and testing sets to ensure proper model evaluation. To reduce the overfitting effect and increase the size of data, data augmentation techniques were applied using Keras' ImageDataGenerator. Data augmentation included rotation, horizontal flipping, random cropping, zooming and brightness adjustment. Furthermore, all of the images were resized to a fixed dimension of 224*224 pixels to match the input requirements of the deep learning models, InceptionV3 and EfficientNetB4.

Step 2: Model Development

After preparing the dataset, building the model started. In building the model, transfer learning came into play with the deployment of two high-level architectures, namely InceptionV3 and EfficientNetB4, that were pre-trained on the vast ImageNet dataset. InceptionV3's original final classification layer as well as EfficientNetB4's final classification layer were removed and replaced with a custom classification head tailored to breed recognition. Specifically, a GlobalAveragePooling2D layer was added to reduce the feature maps into a lower-dimensional space that is then fed into fully connected Dense layers including Dropout regularization to combat overfitting. The output layer was framed with a SoftMax activation function where the

size of output units matched the number of breeds contained within each dataset. In addition to that, models were trained with Adam optimizer with Categorical Crossentropy loss function to suit multiclass classification problems. Training took place with a batch size of 16 over 5 epochs, finding a balance between speed and precision within computational limitations. In their training regime, upper InceptionV3 and EfficientNetB4 layers were tuned while lower convolutional layers remained frozen to maintain inherited general features learned from ImageNet. In such an approach, models were able to effectively adapt to dog and cat breed classification's unique attributes with leverage from pre-trained weight robustness.

Step 3: Model Evaluation

At the end of the training period, developed models were tested with both a validation and a test set to establish their ability to classify dog and cat breeds. Various metrics of performance were implemented, such as overall accuracy alongside per-class accuracy to evaluate individual breed effectiveness. In addition to these metrics, calculations also took place for precision, accuracy for training, accuracy for validation, and recall to give a fuller picture regarding classification effectiveness. To better understand misclassification analysis, a confusion matrix was implemented to highlight most commonly misinterpreted breeds. Finally, comparative InceptionV3 to EfficientNetB4 performance was compared across both a dog and a cat data set to find which architectural design ensured higher accuracy and resilience where fine-grained classification tasks were concerned.

For scenarios where models exhibited overfitting, underfitting, or low accuracy, hyperparameter tuning took place to increase overall performance. Modifications comprised learning rate adjustments of the optimizer to enable more stable convergence, raising epoch number to enable deeper learning where necessary to overcome underfitting, and stricter regularization methods including dropout to prevent overfitting. Batch size adjustments were also implemented to achieve a balance between steadiness in training versus computational expense. Through these iterations, models were able to better capture slight breed details but with robustness across both training and validation data.

Moreover, the assessment also involved applying a prediction function to randomly choose images from the data set, which were then passed through the learned models for prediction. Such an approach allowed a qualitative analysis of the accuracy of the model by comparing its predicted breed labels with ground truth labels of randomly selected samples. Such random

testing presented an informative avenue to confirm the ability of a model to generalise with respect to unseen images as well as highlight specific instances where a model showed good resilience or struggled with breeds that were visually similar.

Step 4: Model Deployment

For deployment purposes, the trained models were converted to TensorFlow Lite (TFLite) format to ensure lightweight yet efficient inference, thereby being suitable for deployment in real-time applications. Thereafter, a Graphical User Interface (GUI) was developed using Tkinter alongside OpenCV to provide an interactive platform to handle breed classification tasks. The developed application is meant to take live images directly from a webcam with the ability to select Dog Breed Mode or Cat Breed Mode depending on a particular classification task. Preprocessing was applied to captured image to match the input requirements demanded by models and then passed to concerned TFLite models for inference. On the GUI screen, predicted breed with its respective accuracy score is projected in real time, thus providing an effective integration of deep learning models within an easy-to-use interface.

2.4 Assumptions or Constraints made

2.4.1 Assumptions

1. The datasets obtained from Kaggle contain correctly labelled images of dog and cat breeds without significant annotation errors.
2. Pre-trained models such as InceptionV3 and EfficientNetB4, originally trained on ImageNet are assumed to generalize well to pet breed recognition tasks.
3. Resizing all of the images to 224*224 pixels preserves the essential breed-specific features required for accurate classification.
4. Data augmentation techniques such as rotation, zooming, cropping, flipping and brightness adjustment are assumed to adequately simulate real-world variations.
5. The selected subset of dog and cat breeds is sufficiently representative for training and evaluating the classification models.

2.4.2 Constraints

1. Only a limited number of breeds were considered, rather than the full set of available breeds due to computational and time constraints. In this project, only 20 dog breeds and 20 cat breeds were considered.
2. The dataset is imbalanced, with some breeds having more training images than others, which may affect the performance. For example, Afghan Hound had 23 training images, Chihuahua only had 14 training images, while Golden Retriever only had 13 training images.
3. Fine-tuning was limited to the top layers of InceptionV3 and EfficientNetB4, while the lower convolutional layers were kept frozen to reduce computational cost.
4. Predictions are restricted to the 20 specific breeds included in the training dataset and the system is unable to classify breeds outside of this scope.
5. The developed GUI application is limited to single image testing instead of large-scale batch classification.

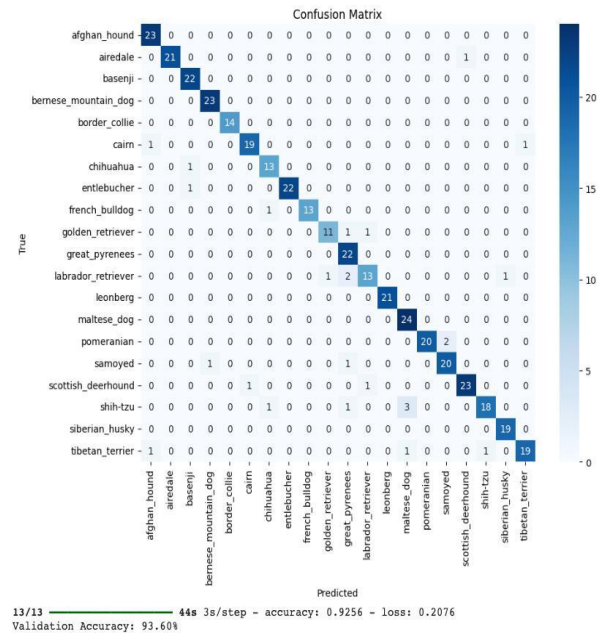
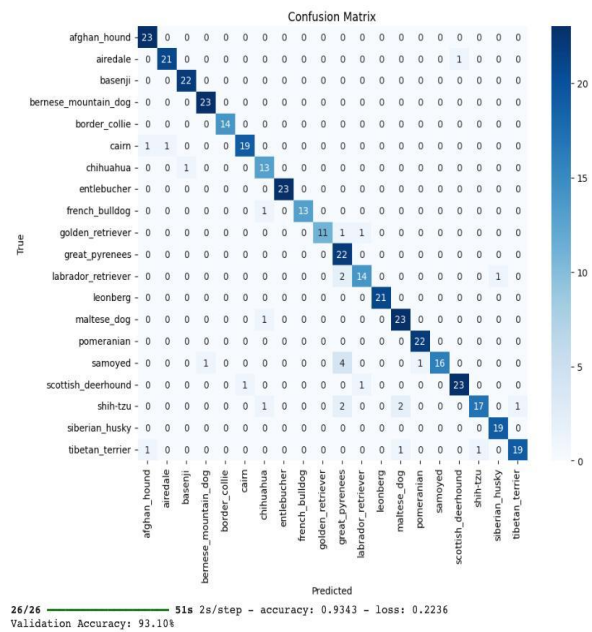
2.5 Result and discussion

2.5.1 Model Performance and Hyperparameter Tuning

The performance of the InceptionV3 model was evaluated by observing their impact on accuracy and generalization. To ensure the model robustness, tuning hyperparameter is indeed an essential measurement. In this case, several experiments were carried out by using different **learning rate, batch size, number of epochs and dropout rate.**

Batch Size Tuning

The effect of batch size on model accuracy was first evaluated. Batch size is a hyperparameter that determines the number of training used in one forward and backward pass of the neural network. With a batch size of 16, the model achieved a 93.1% accuracy while a larger batch size (32) produces a 93.6% accuracy. The left figure below shows the confusion matrix for batch size 16, while the right figure below shows the confusion matrix for batch size 32.



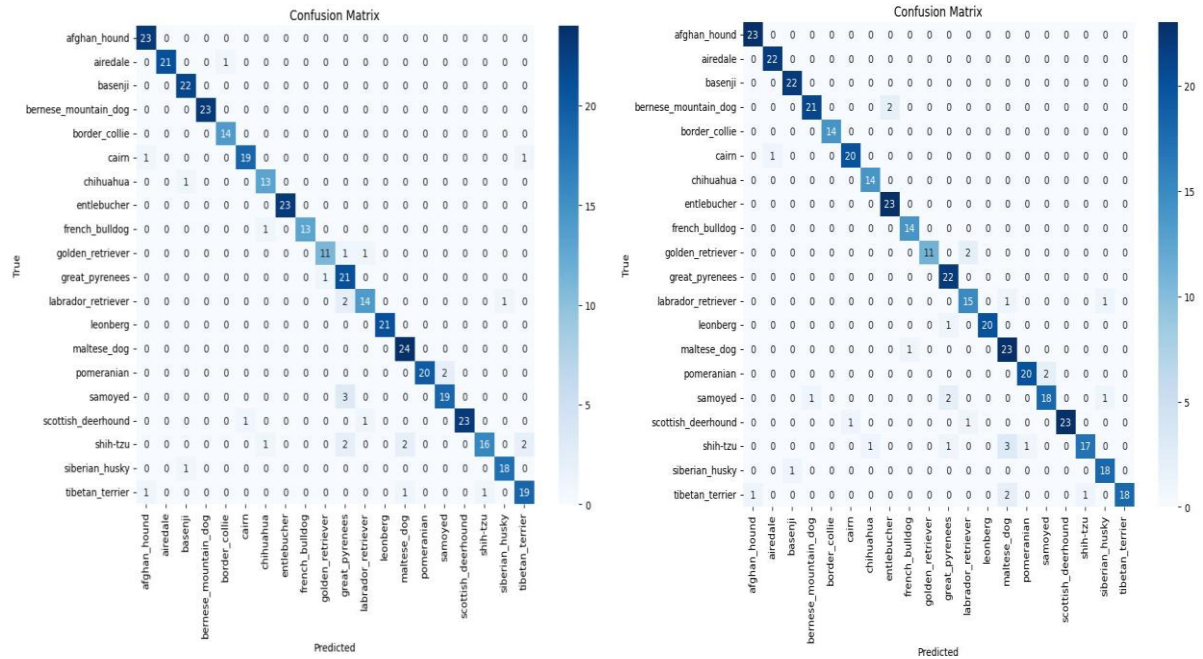
From the confusion matrices shown above, it can be observed that the model trained with a **batch size of 32** correctly classified all instances of the Samoyed breed, while the model trained with a **batch size of 16** misclassified four samples of the same breed. Although there are only minor difference in validation accuracy for both batch size, but we may conclude that larger batch size result in better accuracy especially for breeds with fine-grained similarities.

Number of Epochs Tuning

Epochs refer to one complete pass throughout the entire training dataset. This hyperparameter plays an important role to ensure the model robustness. If the number of epochs are too less, it may lead to underfitting issue. On the other hand, too many epochs may lead to overfitting issue where the model learn the image itself rather than learn the image pattern. In this case, the model is trained and evaluated with normal number of epochs including 10 and 20 epochs to analyse the effect of training duration on performance. With 10 epochs, the model achieved a validation accuracy of 92.86% while extending the training to 20 epochs increased the accuracy to 93.1%. The left figure below shows the classification report of the model with 10 epochs which contains the accuracy, precision, recall and f1-score for every breeds, while the right figure below shows the classification report of the model with 20 epochs.

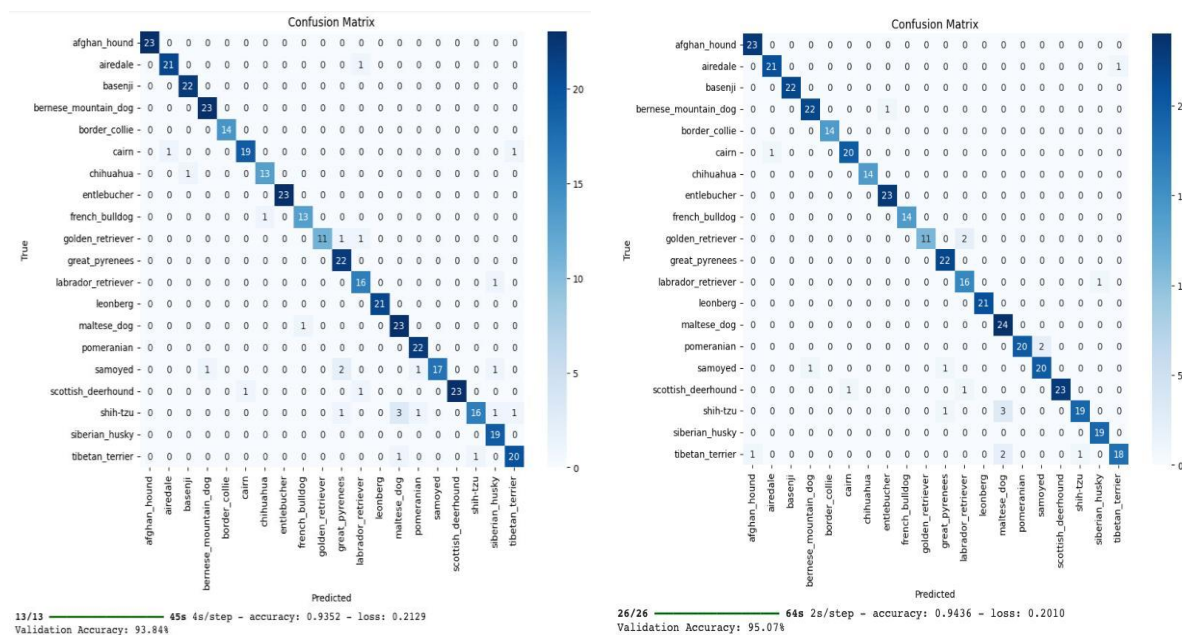
Validation Accuracy: 92.86%					Validation Accuracy: 93.10%				
Classification Report:					Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
afghan_hound	0.92	1.00	0.96	23	afghan_hound	0.96	1.00	0.98	23
airedale	1.00	0.95	0.98	22	airedale	0.96	1.00	0.98	22
basenji	0.92	1.00	0.96	22	basenji	0.96	1.00	0.98	22
bernese_mountain_dog	1.00	1.00	1.00	23	bernese_mountain_dog	0.95	0.91	0.93	23
border_collie	0.93	1.00	0.97	14	border_collie	1.00	1.00	1.00	14
cairn	0.95	0.90	0.93	21	cairn	0.95	0.95	0.95	21
chihuahua	0.87	0.93	0.90	14	chihuahua	0.93	1.00	0.97	14
entlebucher	1.00	1.00	1.00	23	entlebucher	0.92	1.00	0.96	23
french_bulldog	1.00	0.93	0.96	14	french_bulldog	0.93	1.00	0.97	14
golden_retriever	0.92	0.85	0.88	13	golden_retriever	1.00	0.85	0.92	13
great_pyrenees	0.72	0.95	0.82	22	great_pyrenees	0.85	1.00	0.92	22
labrador_retriever	0.88	0.82	0.85	17	labrador_retriever	0.83	0.88	0.86	17
leonberg	1.00	1.00	1.00	21	leonberg	1.00	0.95	0.98	21
maltese_dog	0.89	1.00	0.94	24	maltese_dog	0.79	0.96	0.87	24
pomeranian	1.00	0.91	0.95	22	pomeranian	0.95	0.91	0.93	22
samoyed	0.90	0.86	0.88	22	samoyed	0.90	0.82	0.86	22
scottish_deerhound	1.00	0.92	0.96	25	scottish_deerhound	1.00	0.92	0.96	25
shih-tzu	0.94	0.70	0.80	23	shih-tzu	0.94	0.74	0.83	23
siberian_husky	0.95	0.95	0.95	19	siberian_husky	0.90	0.95	0.92	19
tibetan_terrier	0.86	0.86	0.86	22	tibetan_terrier	1.00	0.82	0.90	22
accuracy			0.93	406	accuracy			0.93	406
macro avg	0.93	0.93	0.93	406	macro avg	0.94	0.93	0.93	406
weighted avg	0.93	0.93	0.93	406	weighted avg	0.94	0.93	0.93	406

The left figure below shows the confusion matrix for 10 epochs, while the right figure below shows the confusion matrix for 20 epochs.

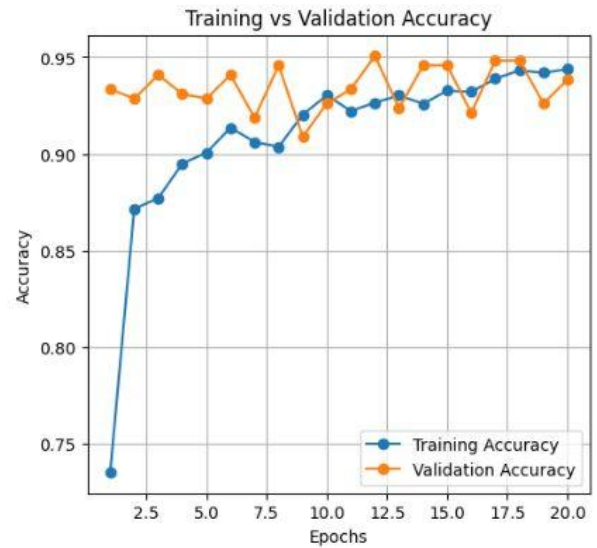
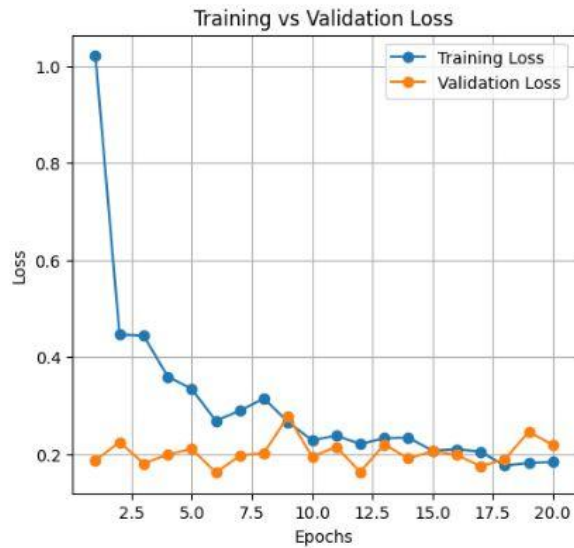


Learning Rate Tuning

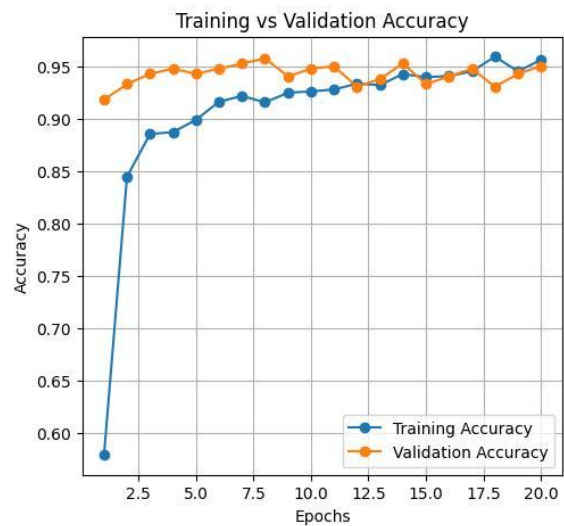
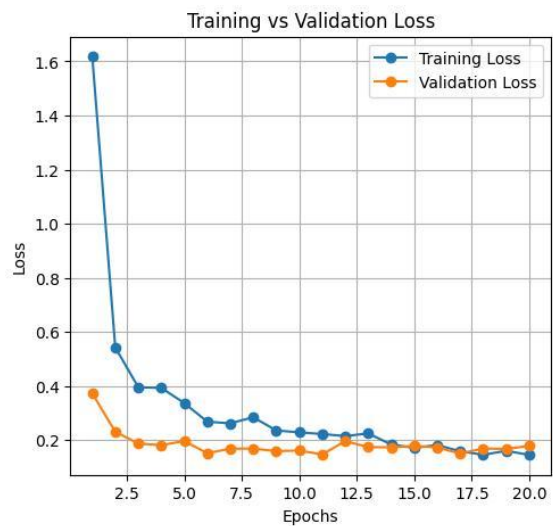
Learning rate is a hyperparameter that controls how quickly the model learns during training. The effect of learning rate on model performance was analysed by testing two values which are **0.001** and **0.0001**. With a learning rate of 0.001, the model converged stably and achieved a validation accuracy of 93.84%. However, when the learning rate was further reduced to 0.0001, the model achieved an even higher validation accuracy of 95.07%, indicating that slower and more precise weight updates allowed the network to extract more discriminative features. The left figure below shows the confusion matrix for learning rate 0.001 while the right figure below shows the confusion matrix for learning rate 0.0001.



From the confusion matrices, it was observed that the learning rate of **0.0001** reduced misclassifications compared to **0.001**, particularly for breeds with fine-grained similarities such as Samoyed and Shih Tzu. Although the model with a learning rate of 0.001 achieved relatively high validation accuracy, it also showed signs of **overfitting**, as illustrated in figure below. In this case, the model memorized the training data rather than learning the underlying patterns, resulting in a large gap between training and validation performance. By contrast, the smaller learning rate of 0.0001 promoted more gradual learning, which improved generalization and reduced the tendency to overfit.



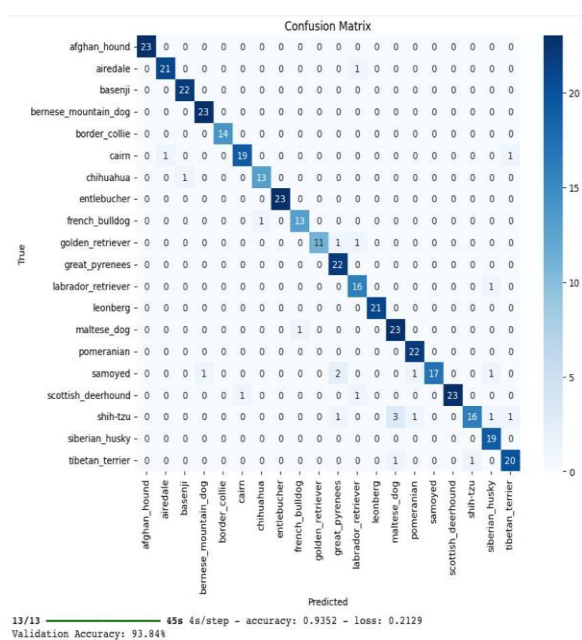
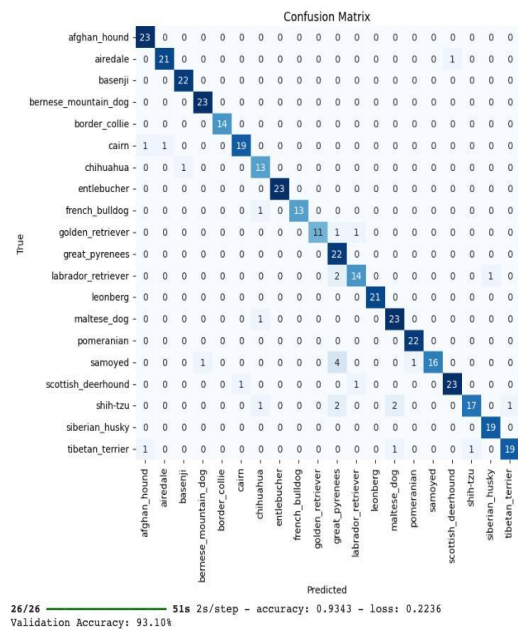
The model with learning rate of 0.001 showed overfitting.



The model with learning rate 0.0001 showed well-fit data.

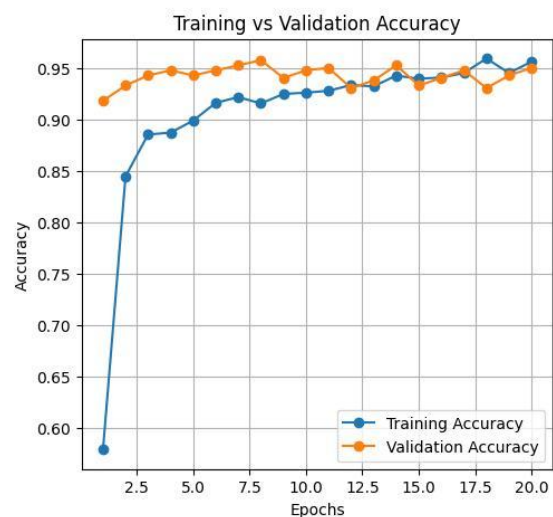
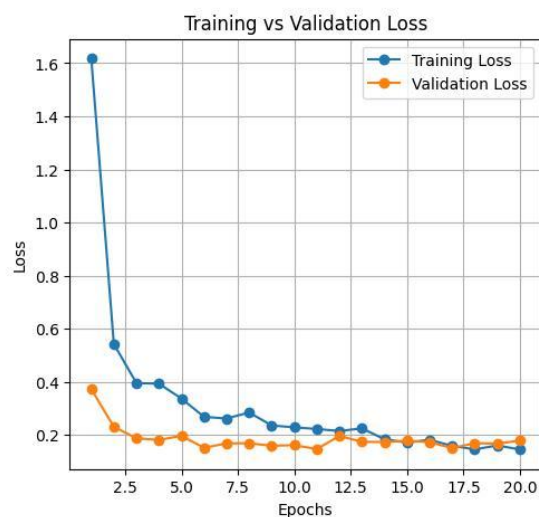
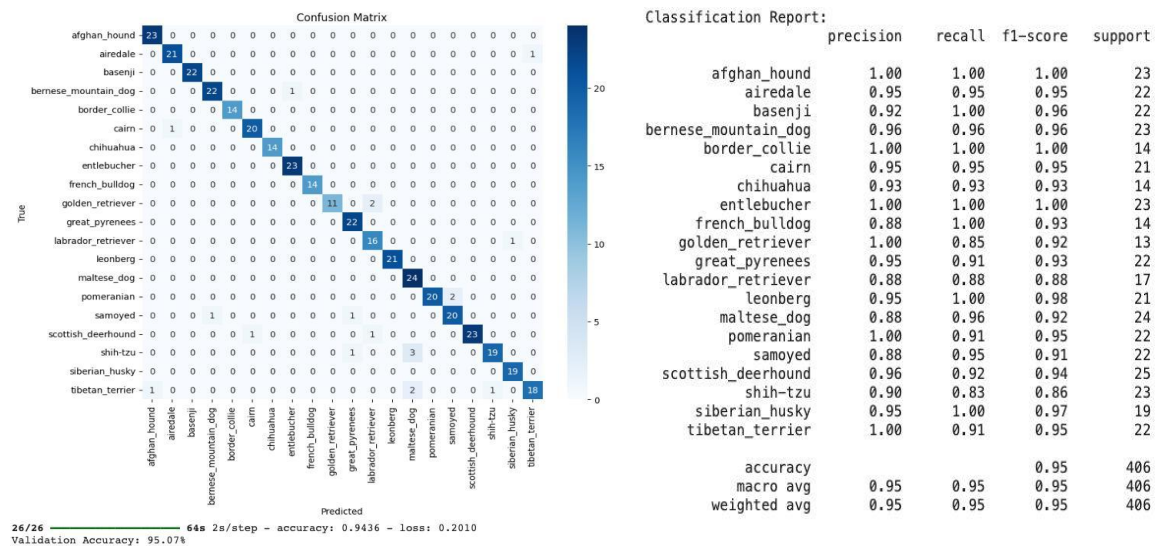
Dropout Rate Tuning

The dropout rate was varied between **0.4** and **0.5** to evaluate its effect on model regularization and classification performance. With a dropout rate of 0.4, the model achieved a validation accuracy of **93.1%**. By increasing the dropout rate to 0.5 slightly reduced training accuracy but improved validation accuracy to **93.84%** demonstrating better generalization. The left figure below shows the confusion matrix for dropout rate of 0.4 while the right figure below shows the confusion matrix for dropout rate of 0.5.

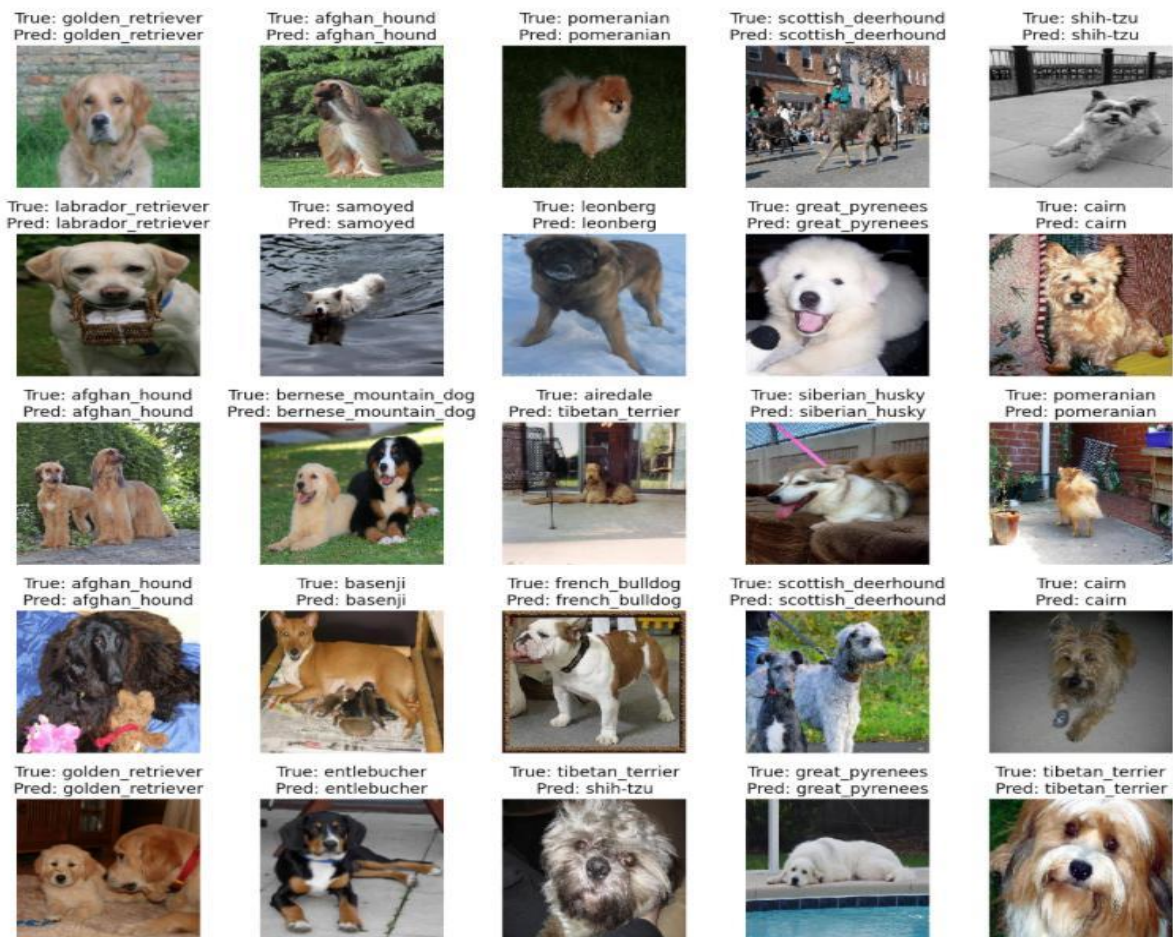


Finalized Hyperparameters and Model Performance

After tuning the hyperparameters, the optimal configuration are as follows, **batch size = 32**, **learning rate = 0.0001**, **epochs = 20** with **early stopping** and **dropout rate = 0.5**. This setup provided the best balance between training and validation performance, minimizing overfitting while ensuring strong generalization. With these finalized hyperparameters, the model achieved a validation accuracy of **95.07%**. As shown in **figure below**, the confusion matrix highlights high per-class accuracy for the majority of breeds, although certain fine-grained breeds such as *Shih-Tzu* and *Tibetan Terrier* were occasionally misclassified due to strong visual similarities. Overall, the final tuned model achieved robust performance.



The model with the finalized hyperparameters set showed well-fit data

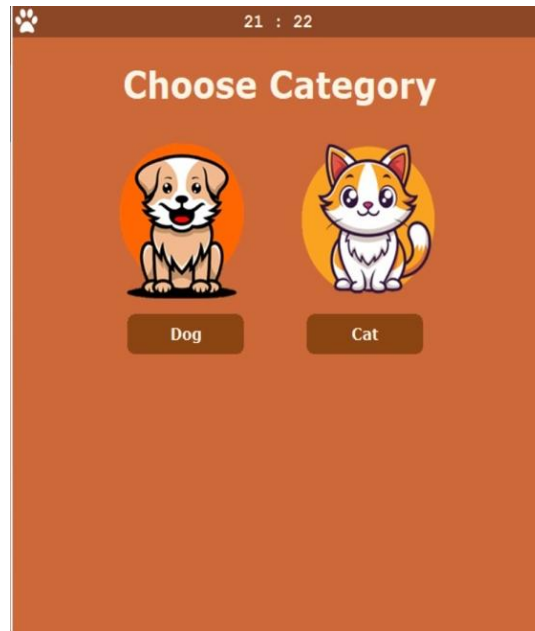


Random prediction of the dog breeds of the finalized model

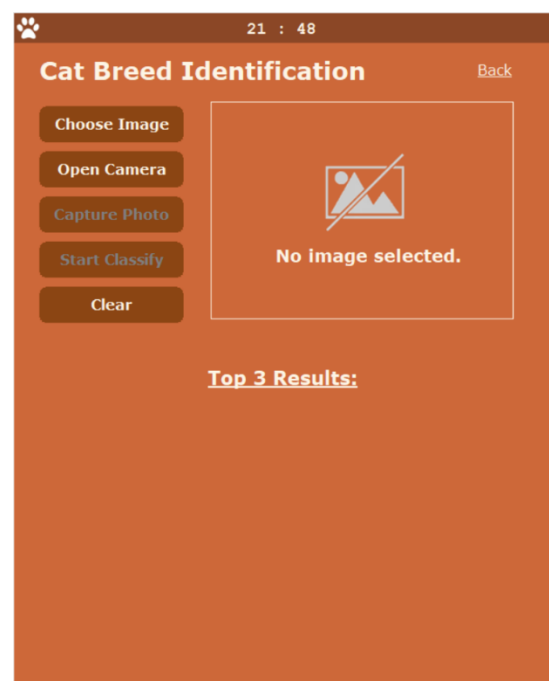
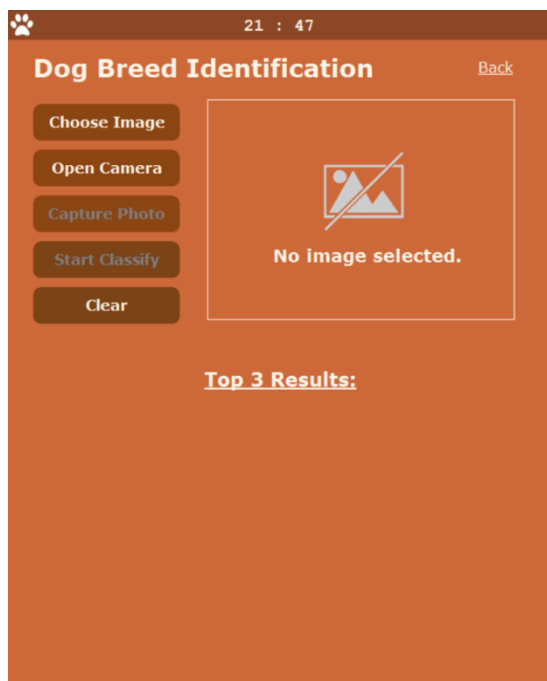
2.5.2 GUI Display

Main Menu

After successfully authenticate the user, a main menu GUI of pet classification above. Users are prompt to choose categories between dog and cat to start identifying

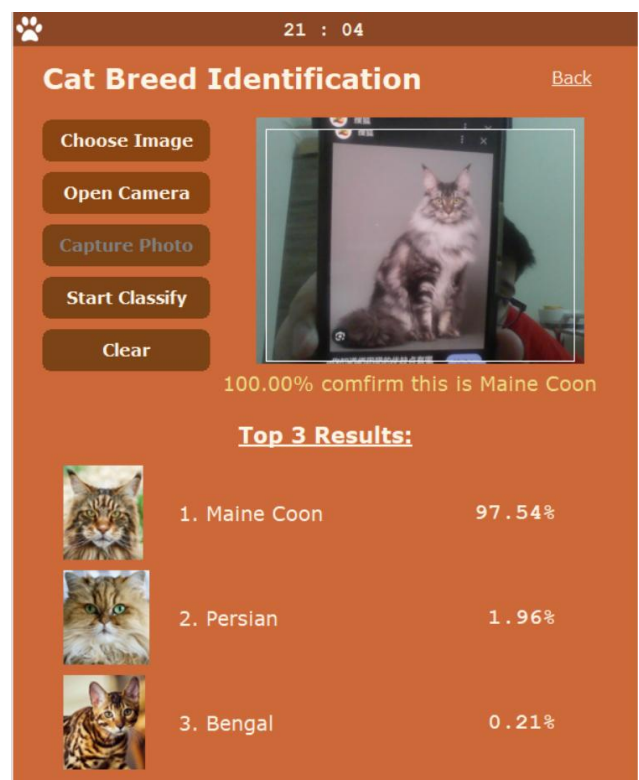


After select one of it, the classification GUI will display. The left figure below shows when dog is selected by the user while the right figure below shows when cat is selected by the user.



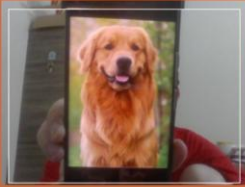
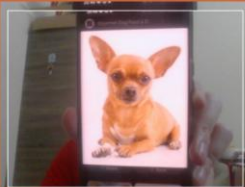
Inside classification GUI, users are given two options to classify the breeds. The first option is to upload an existing image by clicking the **“Choose Image”** button, where user can select the image from their own directory. The second option allow user to capture the image in real-time. The user just need to click the **“Open Camera”** button, then the system activate the webcam for real time capturing live image. Once the image is framed, user can click **“Capture Photo”** to capture the image. After the image is loaded, user can press the **“Start Classify”** button to start the classify the breed belong to the dog/cat image.

To make the GUI more user friendly, different functional buttons are added. First is the **“Clear”** button, which allow user to remove the image after the classification or when the loaded image is incorrect. Other than that, a **“Back”** button is placed at the top-right corner allow users to return to the **Main Menu** without restarting the application.



2.5.3 Classifications Results in Application

Test Case	Expected Breed	Predicted Breed	Result (> 90%)
1	Bombay	Cat Breed Identification Back Choose Image Open Camera Capture Photo Start Classify Clear 100.00% confirm this is Bombay Top 3 Results: 1. Bombay 97.83% 2. Sphynx 1.18% 3. Egyptian Mau 0.24%	Pass
2	Sphynx	Cat Breed Identification Back Choose Image Open Camera Capture Photo Start Classify Clear 100.00% confirm this is Sphynx Top 3 Results: 1. Sphynx 97.24% 2. British Shorthair 1.57% 3. Egyptian Mau 0.38%	Pass
3	Abyssinian	Cat Breed Identification Back Choose Image Open Camera Capture Photo Start Classify Clear 100.00% confirm this is Abyssinian Top 3 Results: 1. Abyssinian 96.23% 2. Bengal 2.48% 3. Egyptian Mau 0.37%	Pass

4	Golden Retriever	Dog Breed Identification Back Choose Image Open Camera Capture Photo Start Classify Clear  100.00% confirm this Is Golden Retriever Top 3 Results:  1. Golden Retriever 99.98%  2. Labrador Retriever 0.01%  3. Great Pyrenees 0.00%	Pass
5	Chihuahua	Dog Breed Identification Back Choose Image Open Camera Capture Photo Start Classify Clear  100.00% confirm this Is Chihuahua Top 3 Results:  1. Chihuahua 99.97%  2. French Bulldog 0.01%  3. Basenji 0.01%	Pass
6	Afghan Hound	Dog Breed Identification Back Choose Image Open Camera Capture Photo Start Classify Clear  100.00% confirm this Is Afghan Hound Top 3 Results:  1. Afghan Hound 99.99%  2. Tibetan Terrier 0.00%  3. Scottish Deerhound 0.00%	Pass

7	Siamese	Cat Breed IdentificationBack Choose Image Open Camera Capture Photo Start Classify Clear  100.00% confirm this is Siamese Top 3 Results:  1. Siamese98.04%  2. Bombay1.00%  3. Egyptian Mau0.24%	Pass
8	Shiba Inu	21 : 11 Dog Breed IdentificationBack Choose Image Open Camera Capture Photo Start Classify Clear  This breed is not in training list Top 3 Results:  1. Basenji21.49%  2. Siberian Husky20.65%  3. Golden Retriever14.65%	Fail
9	America Eskimo	21 : 07 Dog Breed IdentificationBack Choose Image Open Camera Capture Photo Start Classify Clear  100.00% confirm this is Samoyed Top 3 Results:  1. Samoyed99.75%  2. Pomeranian0.11%  3. Great Pyrenees0.07%	Fail

Discussion of Result

Based on the nine test cases conducted, eight produced correct classifications while one resulted in a misclassification. In this case, the image of an **American Eskimo Dog**, was incorrectly identified by the system as a **Samoyed**. This error can be attributed to the strong visual similarities between the two breeds. Both are spitz-type dogs characterized by their white, fluffy coats, erect ears, and similar grooming styles, as well as comparable temperaments and high intelligence. Given these overlapping physical attributes, the system encountered difficulty in distinguishing between the breeds, leading to the misclassification. This highlights a challenge in fine-grained classification, where visually similar breeds require larger and more diverse datasets or additional discriminative features to improve recognition accuracy.

Furthermore, in the eighth test case, the uploaded image (Shiba Inu) belonged to a dog breed that was **not included among the 20 training breeds** used in the model. As a result, the system was unable to provide a valid prediction for the breed. Instead, the model will show the breed that have the same characteristic as Basenji and Husky. This outcome shows a limitation of closed-set classification systems, where the model can only classify the breeds, it has been trained on. When upload an unknown breed, the model will either misclassifies it into one of the known categories or fails to generate a reliable prediction.

2.6 Summary and Future Improvements

2.6.1 Summary

As a summary, this part focused on developing a computer vision system capable of classify dog and cat breeds. Since the dataset obtained from Kaggle is large which consist of 120 breeds and total of 20000+ images, thus only a subset of breeds were selected. To ensure the model generalization, all the pet images are pre-processed through reseizing and augmentation. The classification task was implemented by using transfer learning with pre-trained models. In this case, InceptionV3 and EfficientNetB4 model were used and were fine-tuned for the specific task of breed recognition.

The model is evaluated using performance metrics such as accuracy, precision, recall and confusion matrix to ensure its performance. Other than that, random dog and cat image are also feed to the model to evaluate the classification result. After validating the model performance, the models were finally converted to TensorFlow Lite for lightweight deployment. To simulate

a real world scenario, a user-friendly GUI application was developed using Tkinter, which allows users to upload an image of a cat or dog. The model will start the classification and show top three breeds with its respective confidence score. As a result, this demonstrates the practical implementation of deep learning into an accessible user interface for real-world applications.

2.6.2 Future Improvements

Despite the pet classification model show good result, but it still have area of improvement. As mentioned earlier, one of the limitations of this projects is the size and diversity of the dataset used for training. Despite the current dataset have strong foundation, but it may not fully represent the wide variability of dog and cat breeds in real-world conditions. For example, the images in the datasets are taken under controlled conditions with good lighting or clear background, which makes the classification easier. However, users may capture or upload images in critical environment conditions such as low light, cluttered backgrounds or unusual angles. Therefore, the first improvement that can be made is to preprocess the image. For example, when user upload the image, the system will first preprocess the image by adjusting the image sharpness, contrast and brightness. This measurement aims to reduce noises in the image and increase the accurateness of the result.

Other than that, to make the model more robust, it is also essential to expand the dataset to include more images per breed. This step is crucial to make the model to learn to generalize better. For example, sometime the model may misclassify between two breeds that have similar characteristic such as Tibetan terrier and shih-tzu. To address this problem, one of the approach we can take is increase the dataset, so that the model able to better learn the characteristic between these species.

Last but not least, in the current model, only 20 breeds are used to train the dog model and 9 breeds for cat model. In other word, the model will fail to classify a breed if it is not in the dataset. Therefore, more breeds will be increased in the future to make the model more practical and able to classify various of pet breeds.

References

- [1] A. Castelblanco, E. Rivera, J. Solano, L. Tengana, C. López, and M. Ochoa, "Dynamic face authentication systems: Deep learning verification for camera close-Up and head rotation paradigms," *Computers & Security*, vol. 115, p. 102629, Apr. 2022. doi: 10.1016/j.cose.2022.102629.
- [2] Qi X, Wu C, Shi Y, Qi H, Duan K, Wang X. A Convolutional Neural Network Face Recognition Method Based on BiLSTM and Attention Mechanism. *Comput Intell Neurosci*. 2023 Jan 19;2023:2501022. doi: 10.1155/2023/2501022. PMID: 36711196; PMCID: PMC9879688.
- [3] Golwalkar, R., Mehendale, N. Masked-face recognition using deep metric learning and FaceMaskNet-21. *Appl Intell* **52**, 13268–13279 (2022). <https://doi.org/10.1007/s10489-021-03150-3>
- [4] R. Tripathi, "What are Vector Embeddings?," *Pinecone*, Jun. 30, 2023. <https://www.pinecone.io/learn/vector-embeddings/>
- [5] Z. Ráduly, C. Sulyok, Z. Vadász and A. Zölde, "Dog breed identification using deep learning," *2018 IEEE 16th International Symposium on Intelligent Systems and Informatics (SISY)*, Subotica, Serbia, 2018, pp. 000271-000276, doi: 10.1109/SISY.2018.8524715.
- [6] A. Mugilan, R. Kanmani, V. Prabhu, T. K, R. P. Arun Raj and A. Kiren Karthi, "Enhancing dog breed identification accuracy using hybrid InceptionV3 and Xception model," *2025 3rd International Conference on Advancements in Electrical, Electronics, Communication, Computing and Automation (ICAECA)*, Coimbatore, India, 2025, pp. 1-6, doi: 10.1109/ICAECA63854.2025.11012309.
- [7] R. Srinivasan, S. Iniyar, C. Santhanakrishnan, R. Subash and P. Sudhakaran, "Improving dog breed classification performance through stacked deep learning models with imbalanced data," *2024 International Conference on IoT, Communication and Automation Technology (ICICAT)*, Gorakhpur, India, 2024, pp. 1062-1066, doi: 10.1109/ICICAT62666.2024.10922797.
- [8] C. H. Patil, S. Bhoite, H. Patil and M. Jabde, "Deep learning based cat breed recognition," *Fifteenth International Joint Conference on Advances in Engineering and*

Technology, Feb. 2025, Available:
https://www.researchgate.net/publication/389339621_Deep_Learning_based_Cat_Breed_Recognition

- [9] Y. Feng, "Integrating object detection and deep convolutional neural networks for cat breed classification," *Proceedings of the 1st International Conference on Modern Logistics and Supply Chain Management*, pp. 433-438, 2024, doi: 10.5220/0013337500004558.